

CCGRID 2026

Heterogeneous-Resource-Aware Federated Learning with Intelligent LoRA Allocation and Aggregation

Youye Xie, Yao Lian, Kevin Chen, Abdul Latif, Lingzhi Zhao, Reza Farivar

The Grainger College of Engineering, University of Illinois Urbana-Champaign





Introduction



Federated Learning and LoRA



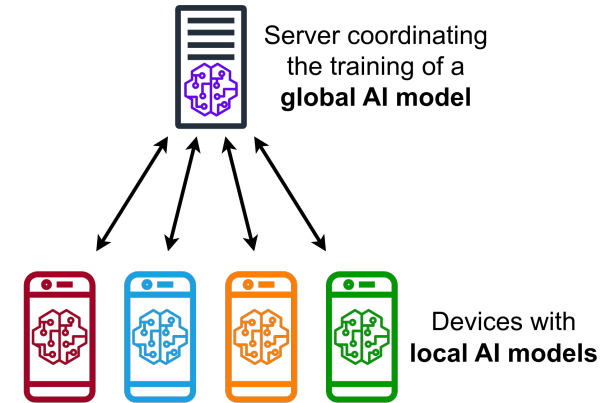
Federated Learning

Enables distributed edge devices to collaboratively train a shared global model while keeping raw data local and private.

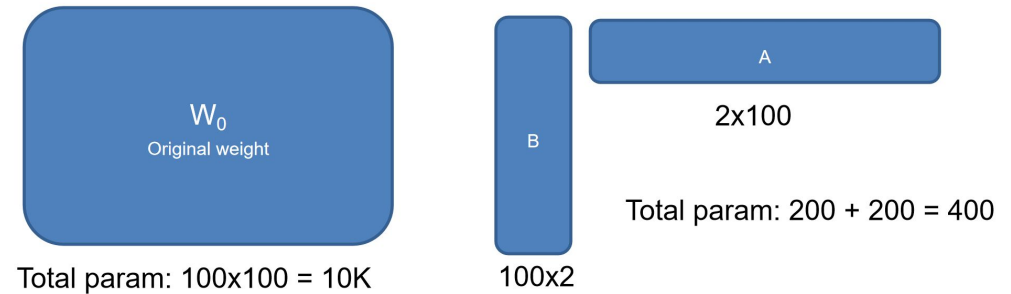


Low-Rank Adaptation (LoRA)

Fine-tunes large models by learning small low-rank adapters.



$$y = W_0 x + B A x \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k} \quad r \ll \min(d, k)$$



Challenges



Communication Overhead

High overhead between server and clients due to the large size of the training model.



Non-IID Data

Non Independent and Identically Distributed (IID) data across clients, hindering global model convergence.



Resource Heterogeneity

Heterogeneous system resources among edge devices, making aggregation and uniform training difficult.

Heterogeneous-Resource-Aware LoRA (HRA LoRA)



LoRA Rank Budget Estimation

Method for each client based on GPU memory and network



Intelligent Rank Distribution

Selects distribution among layers using Fisher Information Matrix (FIM) score



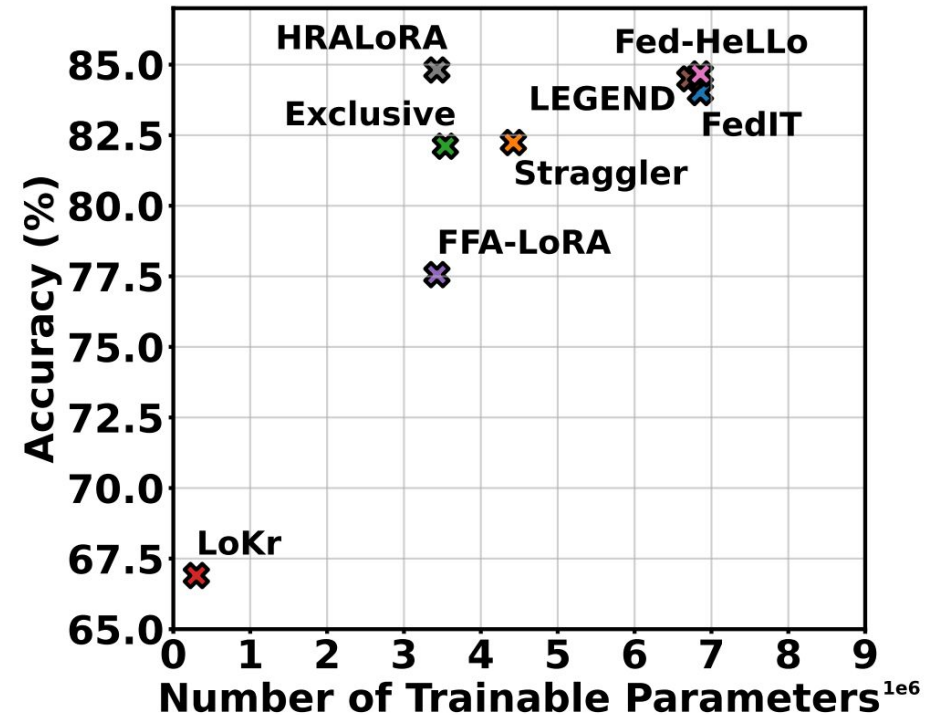
Alternating Training

Reduces trainable parameters and transmitted data by half



SVD-based Aggregation

Balances magnitude between LoRA matrices A and B



Fine-tuning performance on IID CIFAR100 using ViT model *facebook/deit-small-patch16-224*, using the same LoRA rank budget.



HRALoRA



System Overview



LoRA Rank Budget Estimator

Calculates per-client rank budget based on GPU memory and network constraints

Adaptive LoRA Rank Allocation

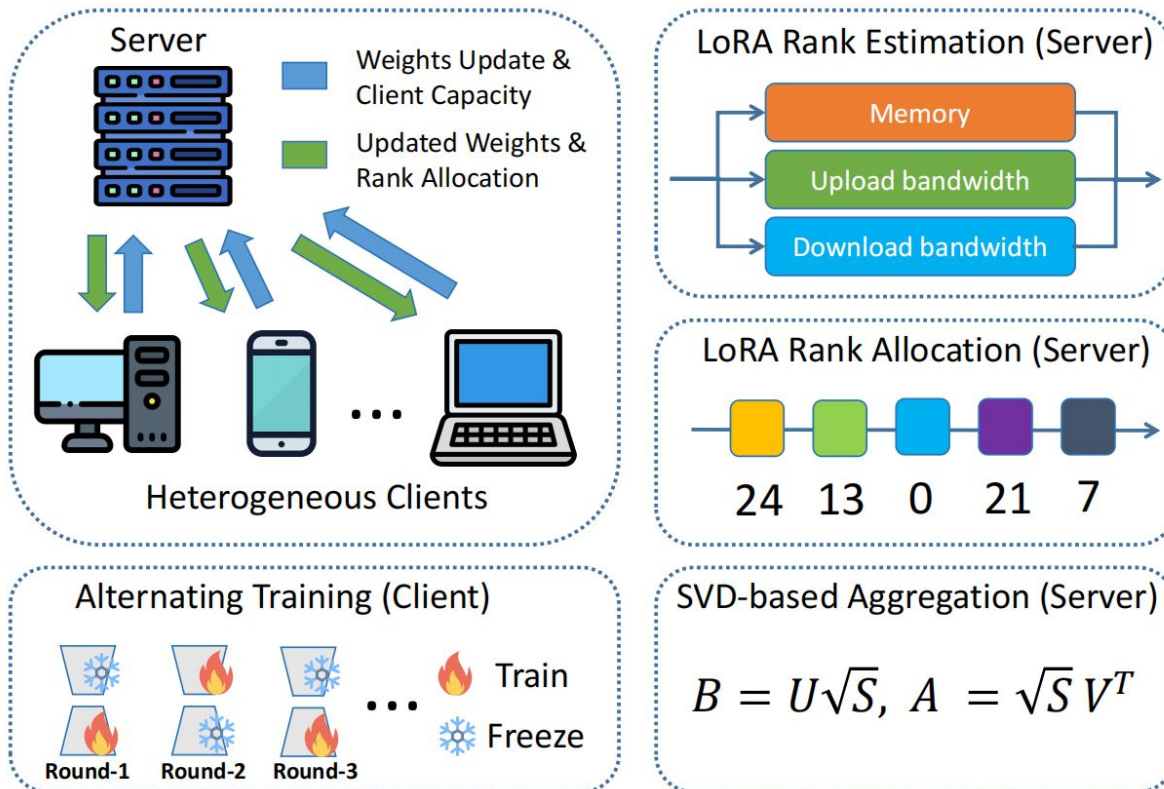
Optimizes rank distribution using FIM scores across layers to improve performance.

Alternating LoRA Training

Improve robustness against non-iid data and reduce trainable parameters and transmitted data by half.

SVD-based Aggregation

Balance magnitude between LoRA matrices A and B for fast convergence.



Rank Budget Estimator



```
rank budget per client = min(r_memory, r_upload, r_download)
```

Memory Constraints

r_memory:

Rank budget per client constrained by GPU memory.

Network Bandwidth

r_upload / r_download:

Rank budget per client constrained by upload and download network bandwidth.



Rank Budget Estimator - GPU Memory

Total GPU Memory Calculation

Total GPU Memory = Sum of the following components:

• Parameters • Optimizer States • Gradients • Forward Pass • Framework Overhead

Memory for Parameters

Count (base + LoRA) * Bytes per parameter

FP32: 4 bytes | FP16: 2 bytes

Memory for Gradients

Backward pass requirement:

LoRA param count * Bytes per parameter

Memory for Optimizer States

Calculation based on LoRA parameters and optimizer type:

LoRA param count * Bytes/param * States (Adam/AdamW: 2, SGD: 1)

Rank Budget Estimator - GPU Memory



Framework Overhead

Memory for framework overhead is estimated by profiling.

Forward Pass Overview

Memory (excluding parameters and overhead):

- Base model part (profiled)
- LoRA part

Fwd Pass LoRA Part

Challenge:

Transformer architecture and framework optimization affect memory for this portion.

Solution:

Our solution can handle this challenge (detailed in the next slide).

Rank Budget Estimator - GPU Memory - Fwd LoRA Part



$$m_{\text{fw_l}} = \text{bytes}_{\text{prec}} \cdot B \cdot S \cdot (\beta_1 \cdot H + \beta_2 \cdot r_{\text{memory}} + e)$$

Key Variables & Definitions

- **B, S, H:** Batch size, sequence length, hidden dimension
- **β_1 , β_2 , e:** coefficients & residual error
- **bytes_prec:** 4 for FP32, 2 for FP16
- **r_memory:** Memory-constrained rank budget

Solution Methodology

- β_1 and β_2 depend on **transformer architecture** and **framework optimization**
- HRA LoRA **automatically** solves for β_1/β_2 via a few profiling runs and **linear regression**
- Final step: Solve for the optimal **r_memory**

Rank Budget Estimator - Network



The Network Equation

$\text{target time} \times \text{upload speed} = \text{bytesprec} \times \text{LoRA parameters}$

(e.g., $1s \times 7.1 \text{ Mbps} = 4 \text{ bytes for FP32} \times \text{LoRA Parameter Count}$)

Upload Constraints

Solve the equation to isolate the rank budget constrained by upload bandwidth:

r_{upload}

Download Constraints

Similarly, solve for download:

r_{download}

Rank Budget Estimator - Final



Allocation Logic

Rank budget per client = $\min(r_memory, r_upload, r_download)$

Derive each client's layer budget **l_budget** by scaling it **proportionally** to its rank budget so avg rank per layer remains the same across clients

Example: Client Resource Distribution

Group	Clients	Mem.	Upload/Download	r_budget	l_budget
1	6	2 GB	7.1 / 51.1 Mbps	288	6
2	6	4 GB	10.6 / 63.4 Mbps	432	9
3	8	8 GB	14.2 / 86.3 Mbps	576	12

* proportionally: $288/6 = 432/9 = 576/12$

Adaptive LoRA Rank Allocation using FIM Score



- Recent advances demonstrate that the LoRA performance could be improved by applying LoRA to layers that are more **sensitive** to the domain dataset.
- To better exploit the heterogeneous resources, we allocate the LoRA to important layers with the number of rank increases **proportional** to its importance/sensitivity.

The **sensitivity** of a layer is measured by **FIM score** on a held-out dataset:

$$FIM_{\text{score}}(W) = \left\| \frac{1}{N} \sum_{i=1}^N (\nabla_W \mathcal{L}_i \odot \nabla_W \mathcal{L}_i) \right\|_F$$

Adaptive LoRA Rank Allocation using FIM Score



General step

- Measure the FIM score of each layer.
- Select a subset of layers equal to the layer budget, prioritizing those with the highest FIM scores.
- Allocate the LoRA rank budget across the selected layers in proportion to their FIM scores.

Algorithm 1 Adaptive LoRA rank allocation

Require: The layer budget l_{budget} , the rank budget r_{budget}

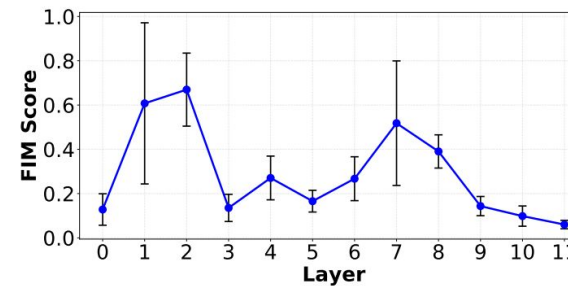
- 1: Initialize an empty list of FIM scores FIM_{list} , the list of selected layers for LoRA $l_{selected}$ and the corresponding list of LoRA ranks $r_{selected}$.
- 2: **for** each layer i in the global model **do**
- 3: Calculate the FIM score $FIM_{score,i}$ following Eq. (9)
- 4: Pushed $FIM_{score,i}$ to the FIM score list FIM_{list}
- 5: Obtain $l_{selected}$ by selecting l_{budget} layers with probabilities generated from FIM_{list} following the layer selection in Fed-HeLLO [18]
- 6: Put the FIM scores of selected layers in a new list $FIM_{selected}$.
- 7: Normalize $FIM_{selected}$ to obtain $P_{rank,i}$ for $i \in l_{selected}$.
- 8: Calculate r_i following Eq. (10) and push it into $r_{selected}$.
- 9: **return** $l_{selected}$ and $r_{selected}$

Adaptive LoRA Rank Allocation using FIM Score

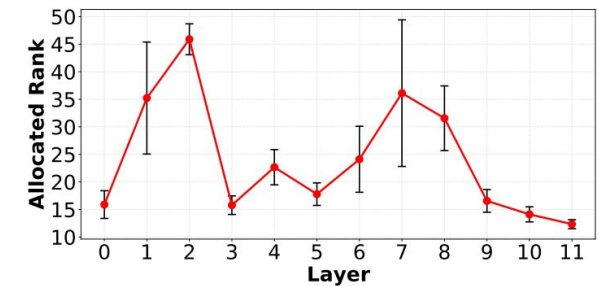


- Layer-wise mean and standard deviation of **FIM scores** and **allocated ranks** when training on IID CIFAR100 data for 200 rounds.

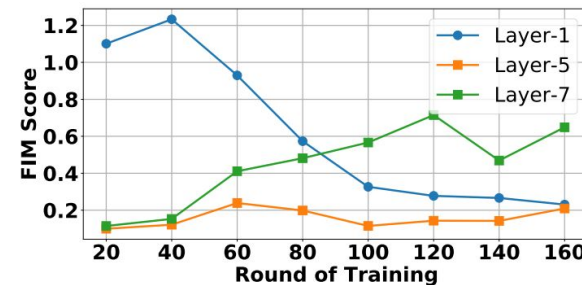
- The big standard deviation in (a) and (b) implies a **sensitivity drift during training**, as shown in (c) and (d) for layer-1 and 7.



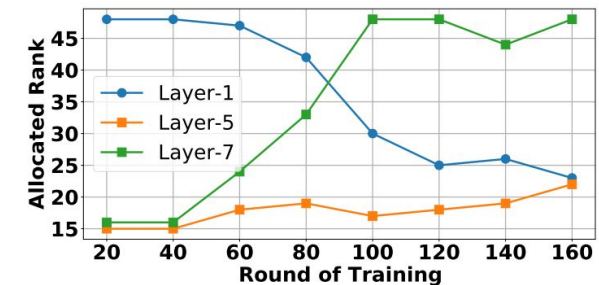
(a) Layer FIM score mean and std



(b) Layer allocated rank mean and std



(c) FIM score in layer 1, 5, and 7.



(d) Allocated rank in layer 1, 5, and 7.

Alternating LoRA Training



- With data heterogeneity, a “**client-drift**” phenomenon [1] is observed, which causes non-stable training and low convergence rate.
- This occurs when the local optimum diverges from the global optimum, a phenomenon that is more **pronounced under non-IID data** when using individual matrix averaging in vanilla LoRA.

$$W = W_0 + \left(\frac{1}{N} \sum_{i=1}^N B_i \times \frac{1}{N} \sum_{i=1}^N A_i \right) \neq W_0 + \frac{1}{N} \sum_{i=1}^N (B_i \times A_i)$$

Vanilla LoRA
Individual avg of matrices

Ideal parameters following
model-averaging

Alternating LoRA Training



Solution

FFA-LoRA [1]

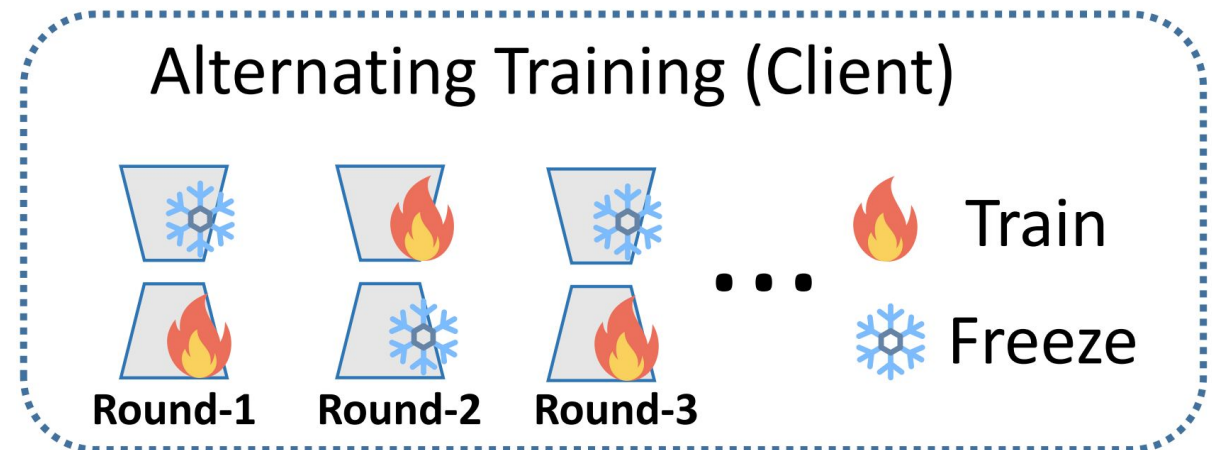
Only train B matrix in LoRA to reduce computational overhead.

HRALoRA

Alternating training approach.

Avoids cases where matrix A (the encoder) projects input to a subspace that fails to represent data effectively.

Alternating Training (Client) Flow



SVD-based Aggregation



- Zero-padding for dimension matching
- SVD-based aggregation ensures that LoRA matrices A and B start with balanced weights after each round of training for improved convergence.

$$B_i A_i = U S V^T, \quad B_{i+1} = U S^{1/2}, \quad A_{i+1} = S^{1/2} V^T$$



Experiments



Experiment Setup - Infrastructure



GPU

NVIDIA H100 (80GB VRAM)

CPU

Intel XEON GOLD 6542Y 24
cores

RAM

128 GB

VM OS

Ubuntu 22.04.5 LTS

DISTRIBUTED TRAINING

Hugging Face Accelerate library



Experiment Setup - Distributed Environment



Simulation

1 Server & 20 Clients
(Following Fed-HeLLo)

Sampling Strategy

5 Clients
Participate Training per Round

Segmentation

3 Heterogeneous Groups

Group	Clients	Memory	Upload / Download Speed
1	6	2 GB	7.1 / 51.1 Mbps
2	6	4 GB	10.6 / 63.4 Mbps
3	8	8 GB	14.2 / 86.3 Mbps

Experiment Setup - Rank Estimator



Divide the 20 clients into 3 groups and **run rank estimator**

Group	Clients	Memory	Upload/Download	r_budget	l_budget
1	6	2 GB	7.1/51.1 Mbps	288	6
2	6	4 GB	10.6/63.4 Mbps	432	9
3	8	8 GB	14.2/86.3 Mbps	576	12

- Required weight transmitting time over network: 1s
- Apply LoRA to "query" and "value" components, for 12-layers model, the average rank budget per layer in group-1 is $288/12/2 = 24$. Same avg rank budget per layer for other groups.

Experiment Setup - Rank Estimator Validation



Validate accuracy of Rank Estimator with **profiling**

Component	Estimated (MB)	Profiled (MB)	Error (%)
Parameters	98.32	98.32 (4.90%)	0.00
Forward Pass	1906.55	1787.53 (89.07%)	6.66
Gradients	14.20	14.20 (0.71%)	0.00
Optimizer States	28.41	28.41 (1.42%)	0.00
Overhead	81.11	78.44 (3.91%)	3.41
Total Peak	2047.48	2006.90 (100.00%)	2.02

Note: In this example, available GPU memory is 2048 MB with a rank budget of 4776. "Error" measures the percentage difference.

Experiment Setup - Dataset



Image Classification Task

Model: facebook/deit-smallpatch16-224

Dataset: CIFAR100

Natural Language Task

Model: google/bert uncased L-12 H-128 A-2

Dataset: LEDGAR

Data Heterogeneity

IID

All clients receive randomly, identically distributed training samples

Non-IID 20

Samples from 20/100 classes, dirichlet distribution for same class across clients

Non-IID 10

Client randomly samples from 10 out of 100 classes



Experiment Setup - Compared Methods

Parameter Reduction

FedIT

Standard FedAvg + LoRA using fixed rank of 24. Activated layers are evenly spaced.

FFA-LoRA

Same as FedIT, but freezes LoRA matrix A which halves trainable parameters.

LoKr

Same as FedIT, but replace LoRA with LoKr for extreme parameter reduction.

Resource Handling

Straggler

Same as FedIT, but bottlenecks all clients to the weakest device's budget.

Exclusive

Same as FedIT, but only top-tier devices train. Weaker clients sit idle if selected.

Layer & Rank Allocation

FlexLoRA

All layers are activated with uniform rank. Weaker client has rank truncated using SVD.

HetLoRA

Same as FlexLoRA, but uses direct matrix truncation and weighted aggregation.

LEGEND

Static rank allocation that assigns higher ranks to later layers.

Fed-HeLLO

Uniform rank in selected layers. Dynamically re-selects active layers using FIM.

Experiment - Image Classification



HRALoRA attains performance on par with or exceeding that of prior LoRA-based FL approaches, with a **small expected upload data size**.

20 Clients 5 selected per round

THE PEFT PERFORMANCE, MEASURED BY ACCURACY ON CIFAR100 USING 20 FEDERATED CLIENTS.

Setting	FedIT	Straggler	Exclusive	LoKr	FFA-LoRA	FlexLoRA	HetLoRA	LEGEND	Fed-HeLLO	HRALoRA
IID Data	84.01%	82.24%	82.11%	66.90%	77.56%	84.00%	83.21%	84.50%	84.67%	84.81%
Non-IID 20	74.29%	69.77%	63.00%	54.59%	56.94%	75.83%	72.71%	76.90%	76.49%	78.83%
Non-IID 10	59.69%	56.97%	49.94%	43.39%	39.58%	67.86%	58.70%	66.88%	62.88%	68.99%
Exp. data size (MB)	6.86	4.42	3.54	0.31	3.43	6.86	6.86	6.72	6.86	3.43

100 Clients 10 selected per round, increased data variance

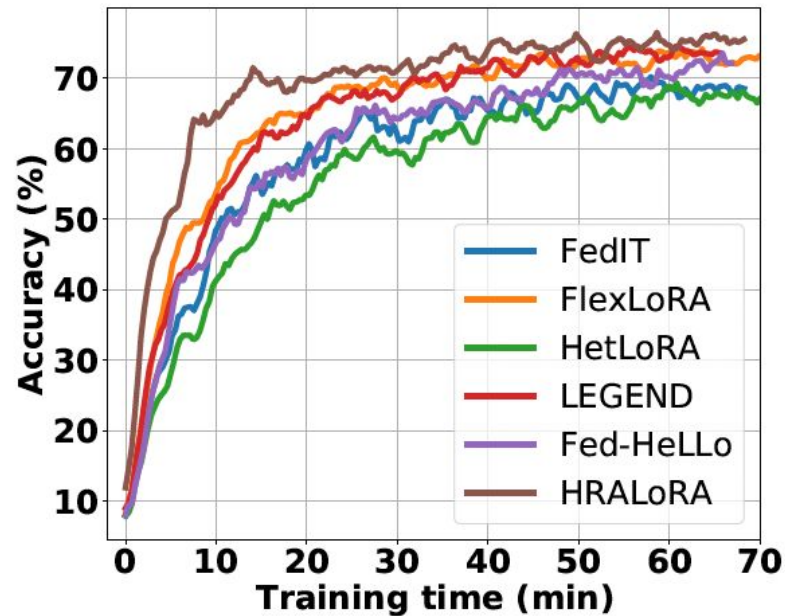
THE PEFT PERFORMANCE, MEASURED BY ACCURACY ON CIFAR100 USING 100 FEDERATED CLIENTS.

Setting	FedIT	Straggler	Exclusive	LoKr	FFA-LoRA	FlexLoRA	HetLoRA	LEGEND	Fed-HeLLO	HRALoRA
IID Data	80.39%	76.10%	78.69%	52.97%	69.35%	79.37%	75.18%	81.42%	81.09%	80.84%
Non-IID 20	72.92%	65.47%	69.71%	36.27%	53.70%	71.15%	63.69%	74.98%	74.24%	76.11%
Non-IID 10	63.10%	51.84%	58.91%	24.66%	36.61%	60.01%	48.22%	66.74%	66.46%	70.05%

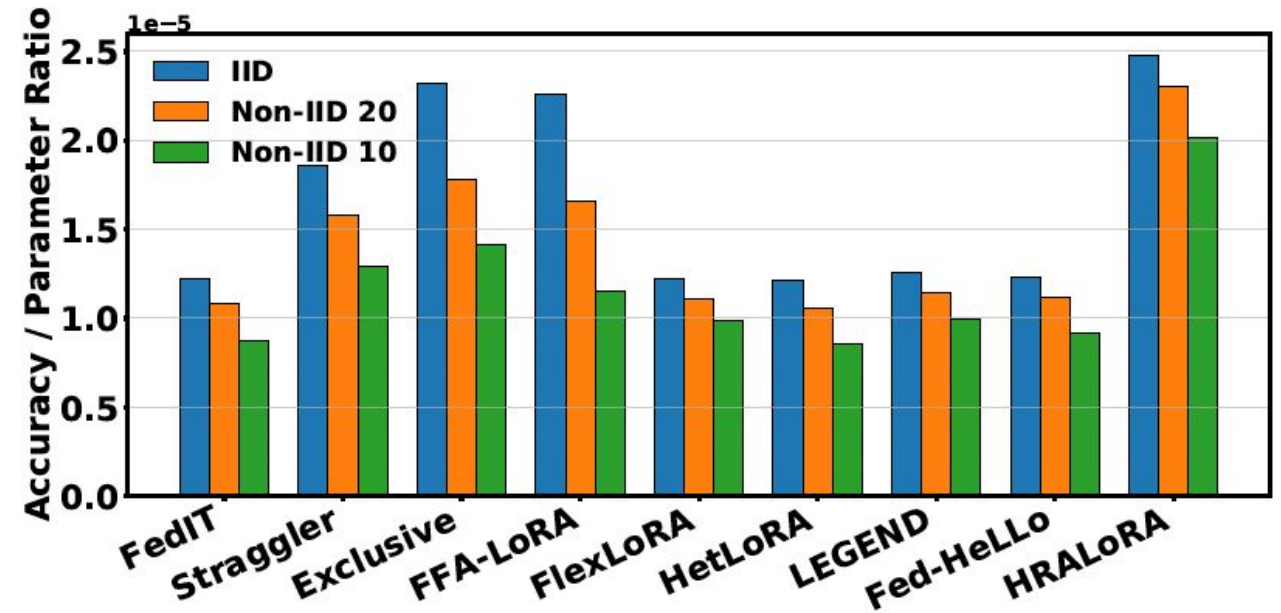
Experiment - Image Classification



Fast convergence and high ratio of accuracy to number of trainable parameters (higher is better)



(a) Time-to-Accuracy performance.



(b) Ratio of accuracy and number of trainable parameters.

Experiment - Text Classification



HRALoRA achieves great performance on text classification tasks as well.

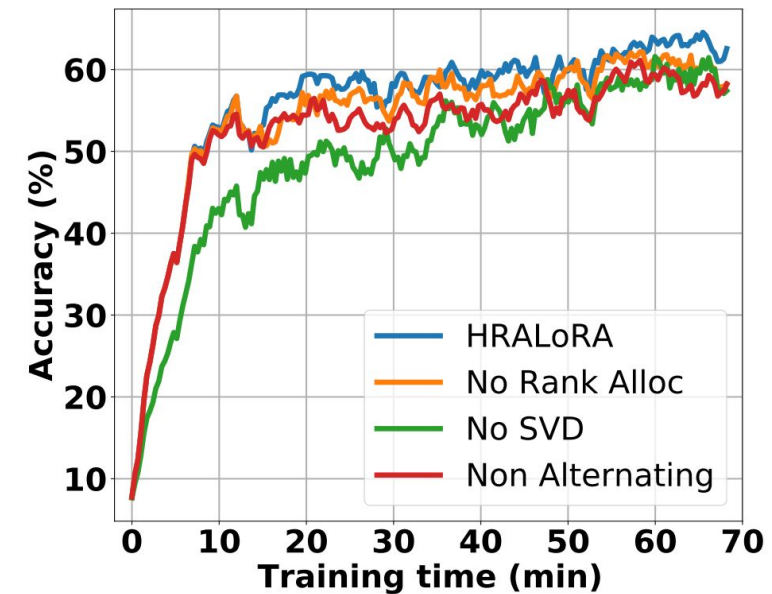
Method	IID Data	Non-IID 20	Non-IID 10
FFA-LoRA	0.685	0.624	0.459
HetLoRA	0.772	0.697	0.541
LEGEND	0.781	0.676	0.549
Fed-HeLLO	0.789	0.701	0.527
HRALoRA	0.775	0.704	0.575

Ablation - Effect of Different Components

- **Dynamic rank allocation** boosts the performance.
- **Alternating training** mitigates “client-drift” under non-IID data and reduces transmitted data.
- **SVD aggregation** brings fast convergence.

ABLATION STUDY SHOWING ACCURACY AND F1 SCORE DROP FOR DIFFERENT COMPONENT REMOVALS WITH NON-IID 10 DATA DISTRIBUTION ON CIFAR100 AND LEDGAR FINE-TUNING.

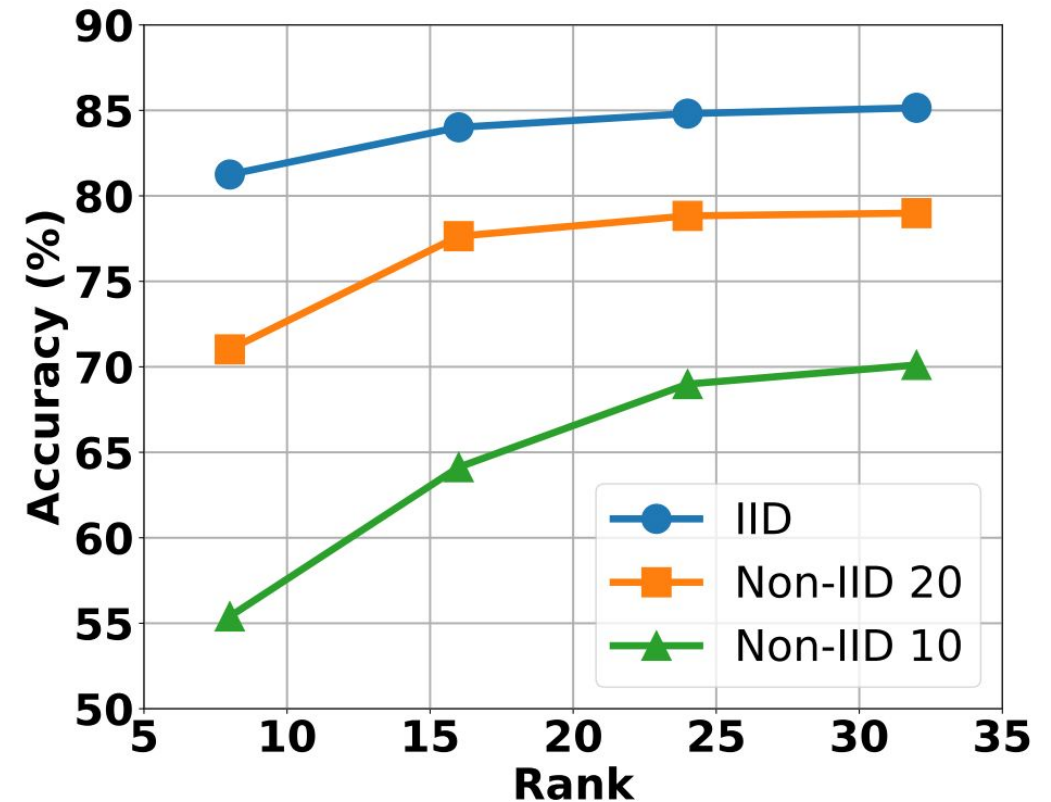
	No-Rank Allocation	No-Alternating	No-SVD
Accuracy Drop	3.02%	4.52%	1.12%
F1 Score Drop	0.0048	0.0173	0.0182



Ablation - Effect of LoRA Rank



- Model performance improves as the average LoRA rank per module increases.
- Beyond 24, further increasing the LoRA rank leads to only marginal performance gains.



Ablation - Effect of Rank Allocation Frequency

- Performance benefits from more frequent rank allocation. But the gain is not linear.
- The impact is most pronounced at high allocation frequencies.

EFFECT OF RANK ALLOCATION FREQUENCY UNDER THE NON-IID 10 SETTING. THE DEFAULT 20-ROUND SETTING ACHIEVES AN ACCURACY OF 68.99% AND AN F1 SCORE OF 0.575 ON CIFAR100 AND LEDGAR FINE-TUNING.

Frequency (rounds)	1	10	20	30
Accuracy Difference	+ 3.71%	+ 1.52%	Ref.	- 0.15%
F1 Score Difference	+ 0.0360	+ 0.0201	Ref.	- 0.0021



Conclusion and Future Work

Conclusion



Novel HRA LoRA Framework

Propose **HRA LoRA**, a **novel LoRA-based federated fine-tuning framework** designed for parameter-efficient learning under heterogeneous client resources.



Constraint Modeling

Explicitly model **memory and communication bandwidth** constraints to characterize feasible LoRA ranks and develop a rank budget estimator.



Optimized Strategies

Fisher Information Matrix (FIM) score-based rank allocation for better performance, **alternating training** for non-IID data robustness, and **SVD aggregation** for fast convergence.



Experimental Validation

Extensive **image and language** fine-tuning experiments under **IID/non-IID** settings demonstrate the **effectiveness and efficiency** of HRA LoRA.

Future Work



Compute Heterogeneity

Variations in FLOPS, TOPS, and HBM bandwidth.



Energy-Aware Rank

Considers energy constraints and performance contribution.



Asynchronous

Extension of HRALORA: from sync to async aggregation.



Multi-Modal Models

Extend evaluation on multi-model models like audio applications.



Robustness Against Malice

Protects against clients transmitting misleading profiles and weights.

I ILLINOIS